

... REPORTE DE ACTIVIDAD 11

Nombre del proyecto: File path traversal, validation of start of path

Nivel: Apprentice

Nombre: Karla Vanessa Glz Espinoza

TIPO DE EXPLOTACIÓN

Path Traversal (Directory Traversal) por validación insuficiente de prefijo / CWE-22: Improper Limitation of a Pathname to a Restricted Directory. Path Traversal (Directory Traversal) Bypass / CWE-22: Improper Limitation of a Pathname to a Restricted Directory.

OBJETIVO DEL LABORATORIO

Comprender las deficiencias de implementar validaciones de seguridad basadas únicamente en el prefijo (inicio) de una ruta de archivo y demostrar cómo la falta de canonicalización permite a un usuario acceder a archivos fuera del directorio restringido.

EXPLICACIÓN TÉCNICA

Una estrategia defensiva común (y defectuosa) contra los ataques de Path Traversal consiste en verificar si el nombre de archivo proporcionado por el usuario comienza con una ruta base esperada. Por ejemplo, un desarrollador podría usar funciones como `startsWith("/var/www/images/")` en Java o `strpos(path, "/var/www/images/") === 0` en PHP.

El fallo crítico de este enfoque radica en que el sistema operativo procesa las secuencias de navegación de directorios (`../`) de forma dinámica. Si un usuario proporciona una ruta que satisface la validación del prefijo, pero inmediatamente después anexa secuencias de "salto hacia atrás" (directory climbing), el filtro de seguridad lo considerará válido (MITRE, 2023).

De acuerdo con CISA (2022), validar solo la forma original de una cadena de texto sin interpretarla tal como lo haría el sistema de archivos subyacente conduce a la evasión de los controles de aislamiento. La aplicación evalúa la cadena literal y la acepta, pero el sistema operativo resuelve las secuencias `../`, escapando del directorio restringido y comprometiendo la confidencialidad de la información (OWASP Foundation, 2023).

PROCEDIMIENTO

Paso 1: Se inició el entorno virtual del laboratorio provisto por Web Security Academy y se configuró Burp Suite Community Edition para la intercepción de tráfico HTTP.

Paso 2: En la sección del historial HTTP (HTTP History) de Burp Suite, se identificó la solicitud que obtiene los recursos de imágenes. Se observó que el parámetro de la ruta requería un directorio específico.

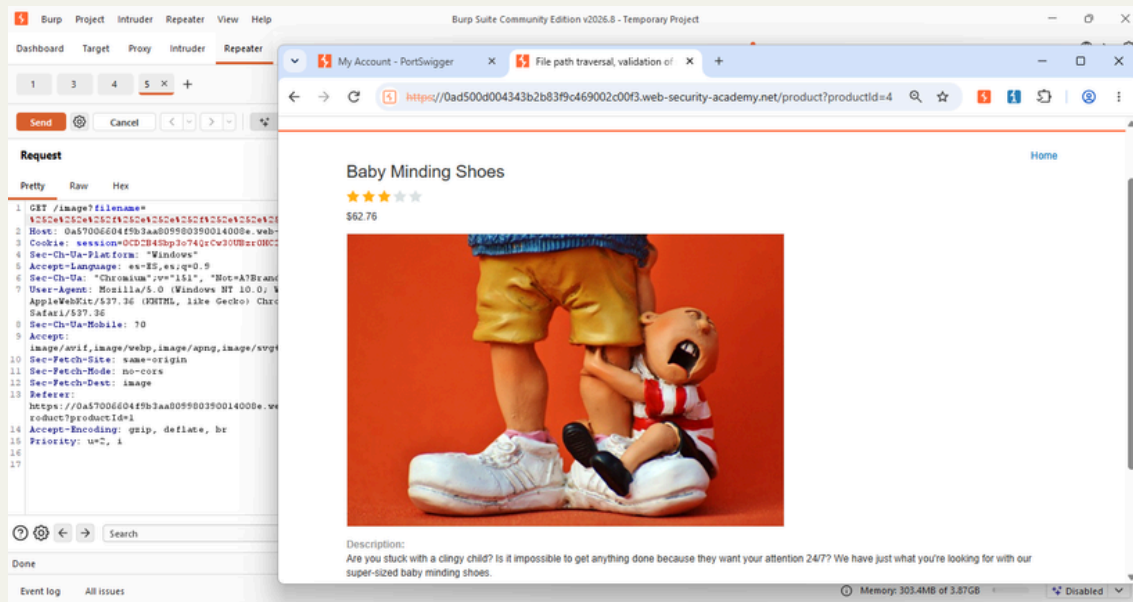


Figura 11.1 Pagina principal de la tienda en un apartado de “View details” de un producto.

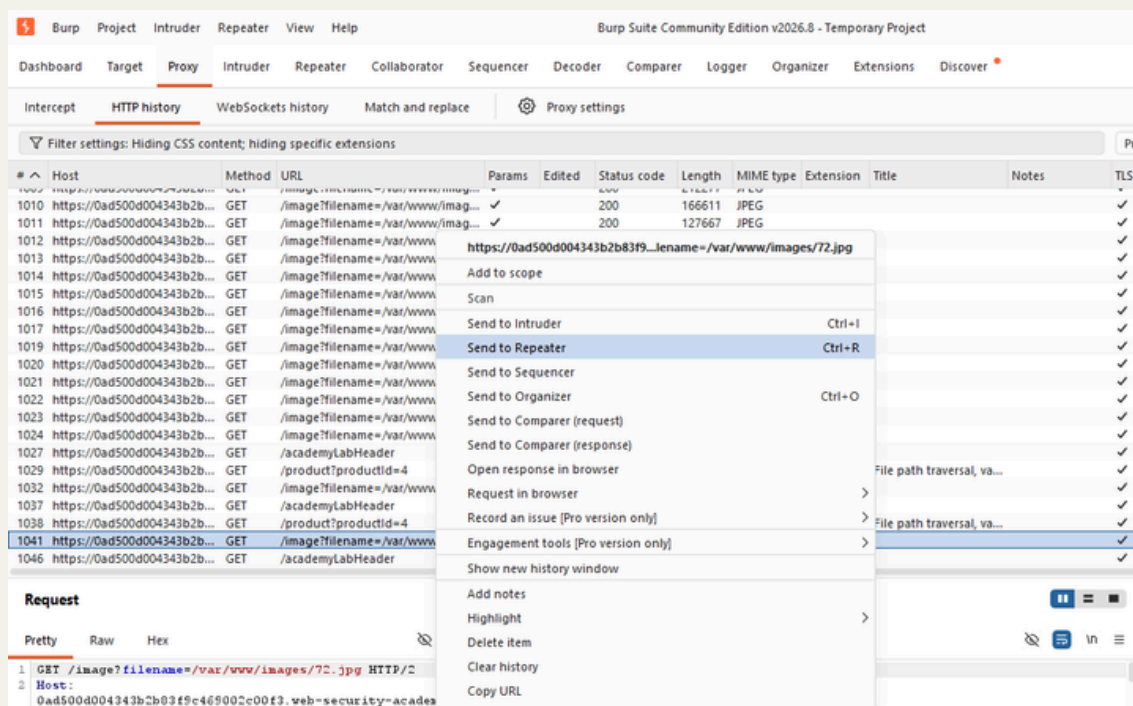


Figura 11.2 Se envió la petición a Repeater para su manipulación

Paso 3: Se envió la solicitud al módulo Repeater para realizar pruebas de validación.

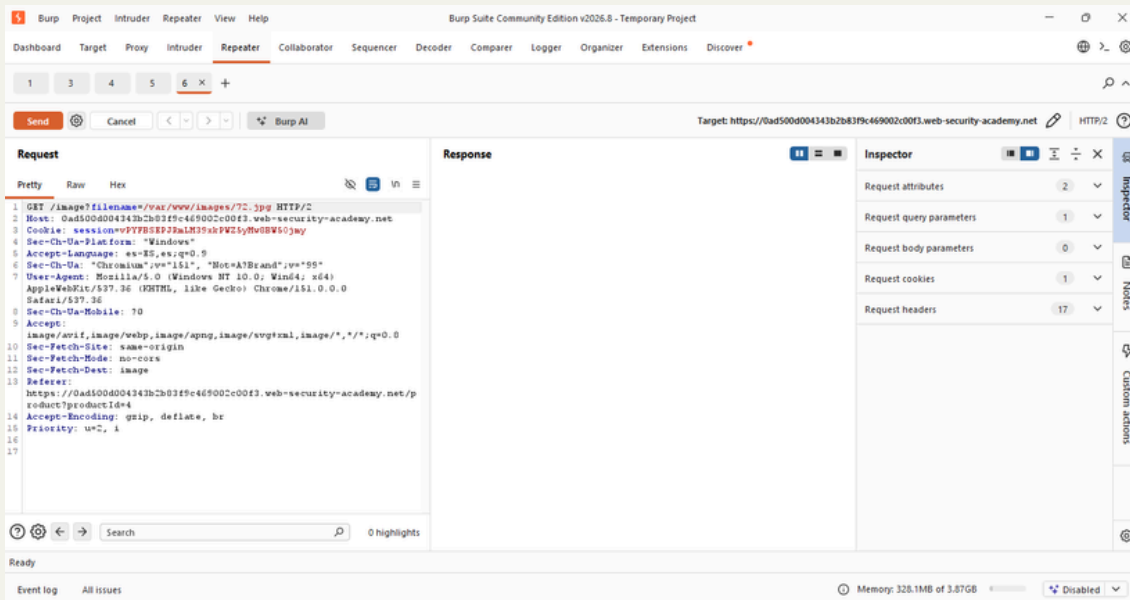


Figura 11.3 Parámetro filename original

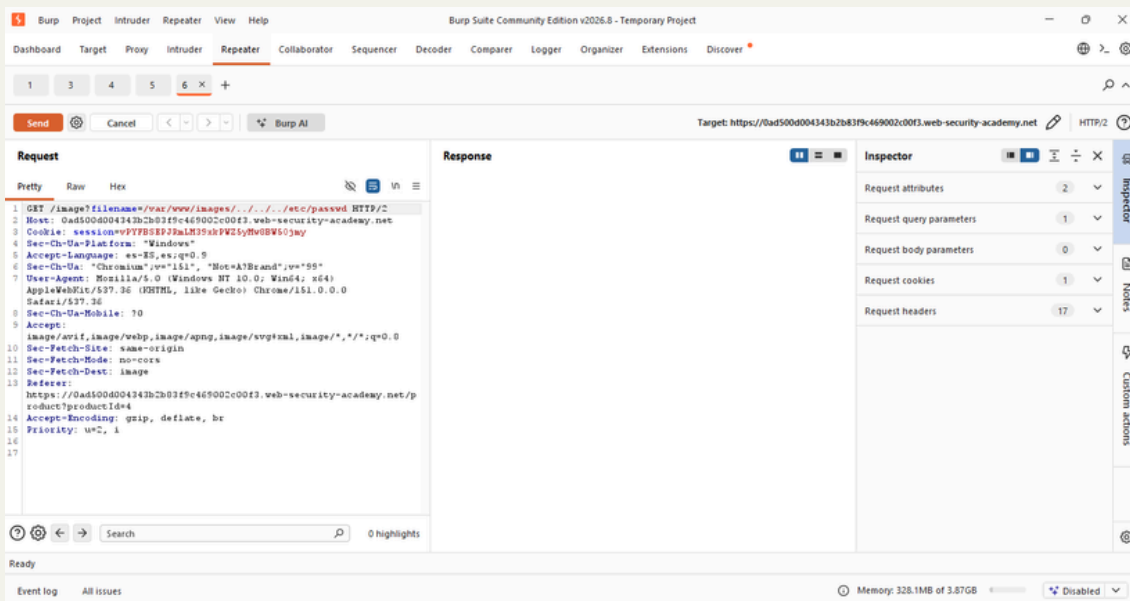


Figura 11.4 Parámetro filename modificado

Paso 4: Se demostró la deficiencia del filtro inyectando un patrón de salto de directorios inmediatamente después de la ruta inicial requerida, observando la resolución exitosa por parte del sistema operativo.

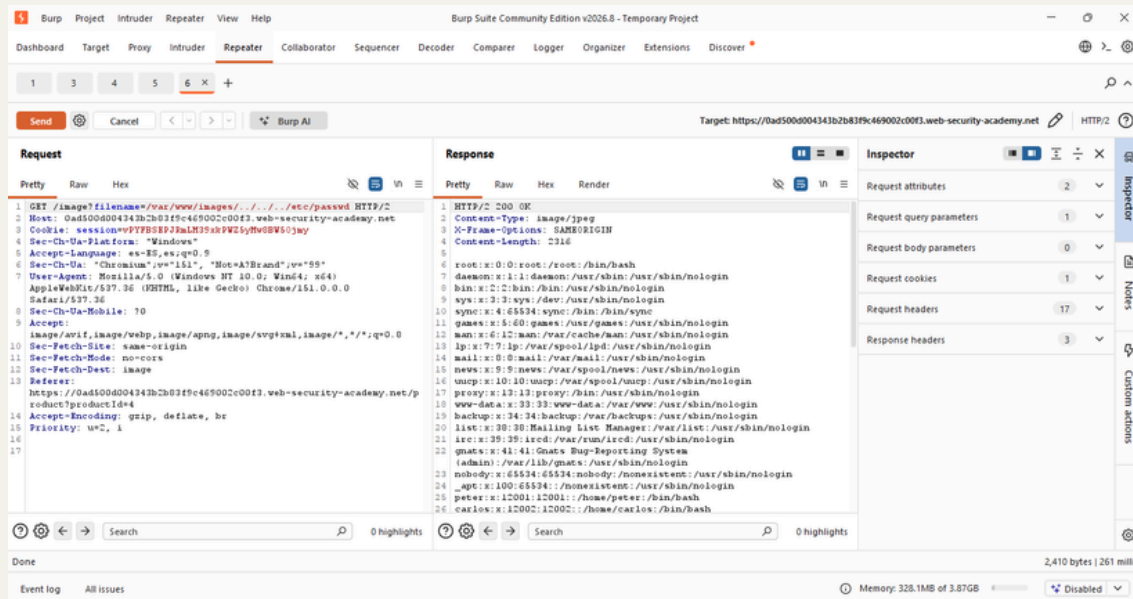


Figura 11.5 Respuesta del servidor al filename modificado

Paso 5: Se analizó la respuesta HTTP, la cual devolvió el contenido de un archivo restringido, validando teóricamente la vulnerabilidad.

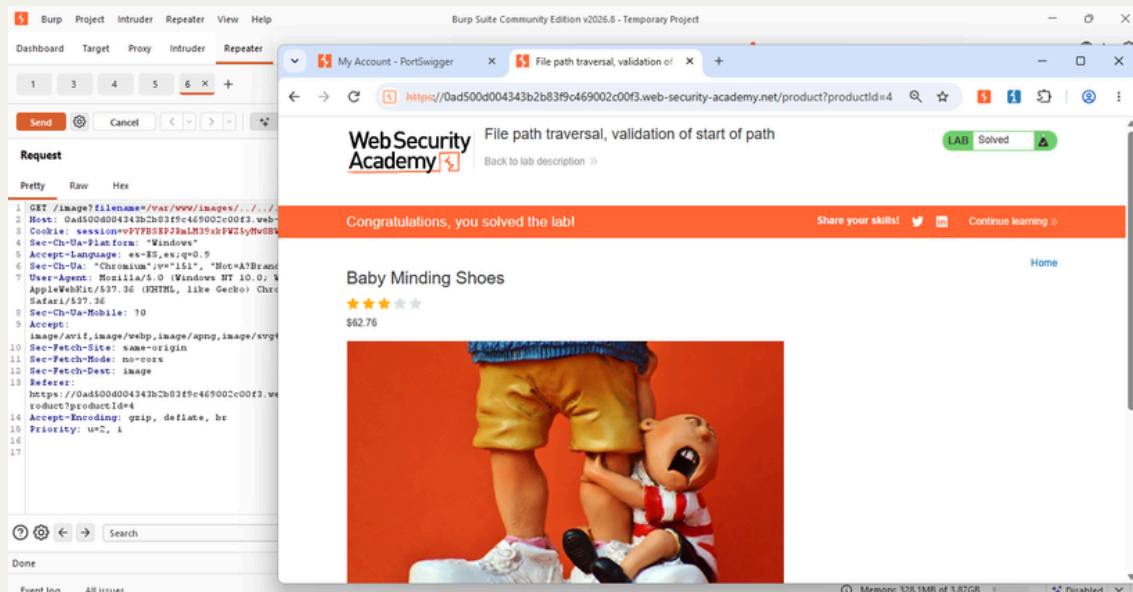


Figura 11.6 Laboratorio finalizado

EXPLICACIÓN DEL PAYLOAD

Cuando una aplicación valida una cadena comprobando que empieza con [DIRECTORIO_SEGURO], la cadena [DIRECTORIO_SEGURO]/../../[ARCHIVO_SENSIBLE] pasará la comprobación lógica porque la condición de texto se cumple (el prefijo es correcto). Sin embargo, al entregar esta misma cadena a una API del sistema de archivos, el sistema primero entra a [DIRECTORIO_SEGURO] y luego usa los ../../../ para retroceder niveles hasta llegar a la raíz, dirigiéndose finalmente al [ARCHIVO_SENSIBLE]. Esto demuestra por qué el análisis léxico de cadenas no es un mecanismo de seguridad válido para rutas de archivos.

MITIGACIÓN RECOMENDADA

Para remediar esta vulnerabilidad, las defensas deben basarse en la ruta real que el sistema operativo accederá, no en la cadena de texto original (OWASP Foundation, 2023).

- Canonicalización de rutas: El paso más crítico es resolver la ruta absoluta real utilizando funciones seguras proporcionadas por el entorno (como `realpath()` en PHP, `os.path.realpath()` en Python o `Path.GetFullPath()` en C#). Estas funciones resuelven todos los enlaces simbólicos y secuencias `../`.
- Validación post-canonicalización: Una vez obtenida la ruta canónica, la aplicación debe verificar explícitamente que dicha ruta comience con el directorio base autorizado.
- Mapeo indirecto (Defensa en profundidad): La mejor práctica sigue siendo evitar por completo que el usuario manipule rutas de archivos. Se deben utilizar identificadores únicos (como un UUID o un ID en la base de datos) para hacer referencia a los archivos (CISA, 2022).

REFERENCIAS

- CISA. (2022). Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). Cybersecurity and Infrastructure Security Agency. <https://www.cisa.gov/uscert/bsi/articles/knowledge/coding-practices/path-traversal>
- MITRE. (2023). CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>
- OWASP Foundation. (2023). Path Traversal. Open Worldwide Application Security Project. https://owasp.org/www-community/attacks/Path_Traversal